- BUZZWORDS-DESIGN GOALS
  - Key Design Goals
  - DESIGN GOALS :Scalability
    - Scaling Up
      - Importance of Scalability
- DESIGN GOALS :Availability
  - Example
  - High Availability
- DESIGN GOALS :Consistency -Strong or Eventual
  - Strong Consistency
  - Eventual Consistency
  - DESIGN GOALS :Fault Tolerance and Single Point of Failure
    - What is SPOF?
    - Avoiding SPOF
    - Example of a Fault Tolerant System:

BUZZWORDS-DESIGN GOALS

In this section, we'll cover key goals to always keep in mind when designing any system:

Key Design Goals

- 1.Scalability
- 2.Availability
- 3.Consistency
- 4.No Single Point of Failure (SPOF)
- 5.Fault Tolerance

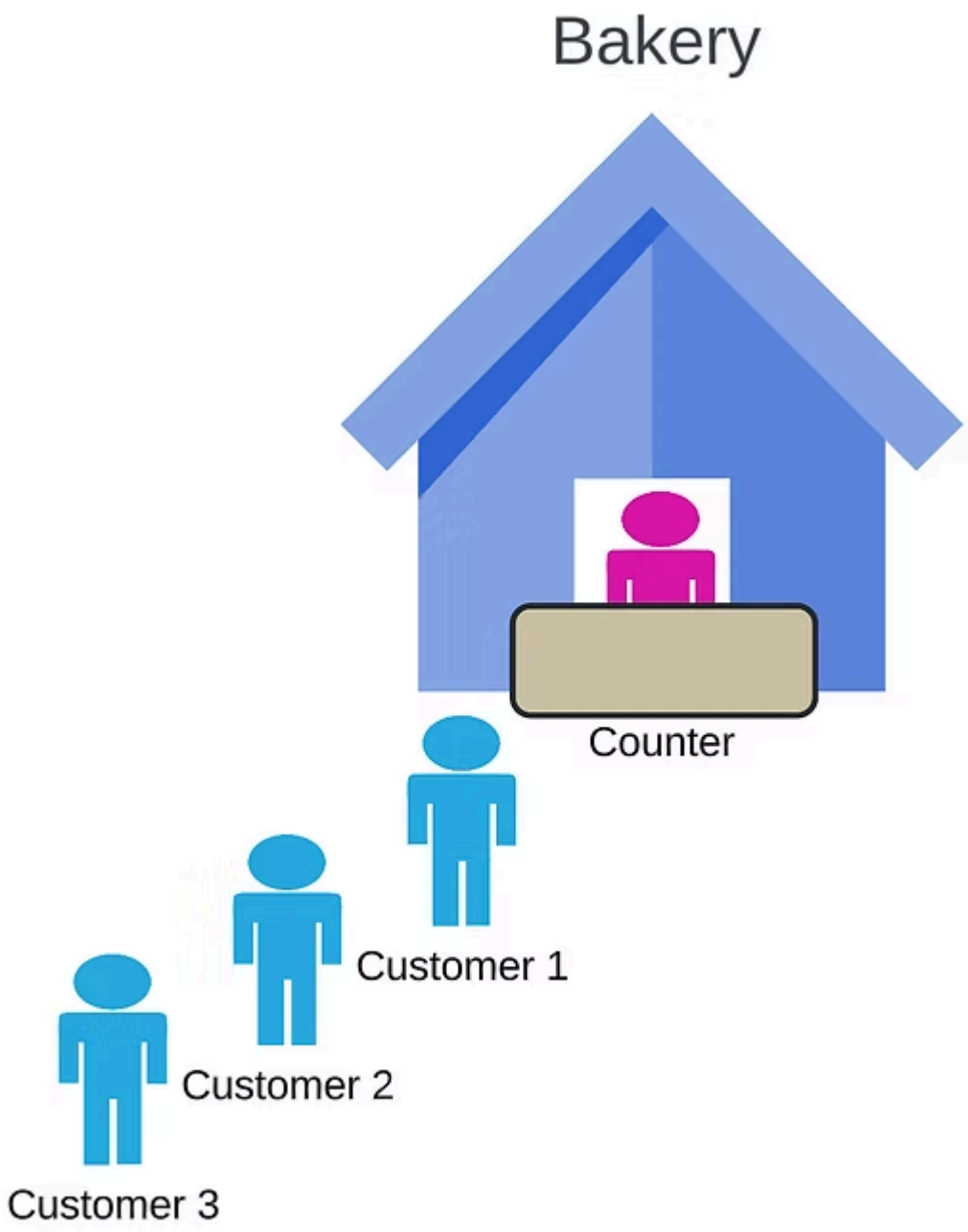
When you prepare food, no matter the ingredients or recipe, you want it to be **tasty, healthy, and affordable**.
Just like these qualities make a great meal, these goals make a great system.

DESIGN GOALS :Scalability

Imagine a local bakery that initially handles its customers with just one cashier.

Now the bakery becomes more popular.

Because of that, the line of customers grows longer, and waiting times increase.



Scaling Up

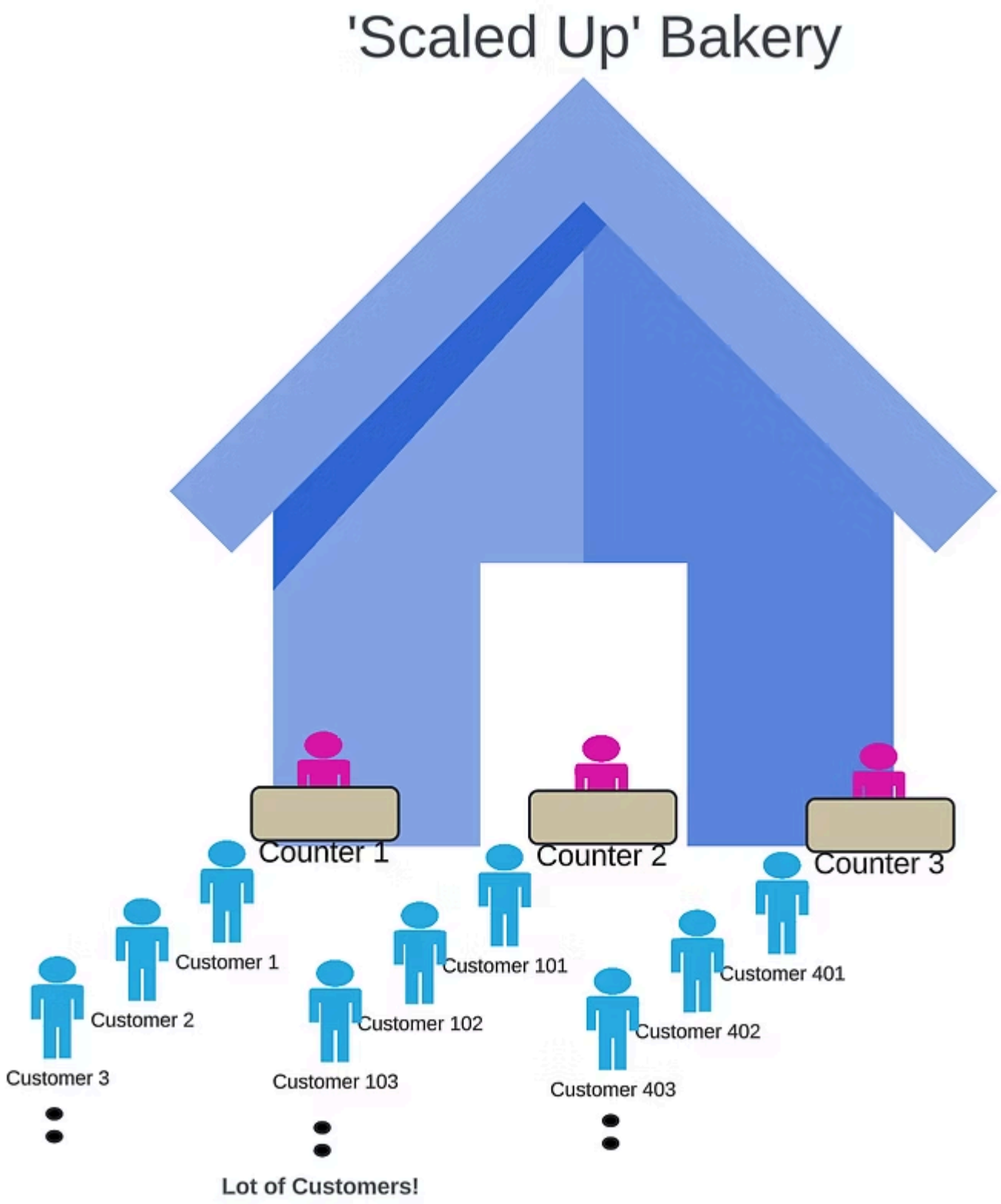
To serve more customers, the bakery opens additional checkout counters, helping them handle the growing crowd efficiently.

Basically, our bakery has **scaled up** to meet increased demand. Just like the bakery, our technical systems also need to **scale up** as more users join.

Importance of Scalability

A well-designed system can scale up more easily.

Scalability is the system's ability to handle more work smoothly as demand grows.



DESIGN GOALS :Availability

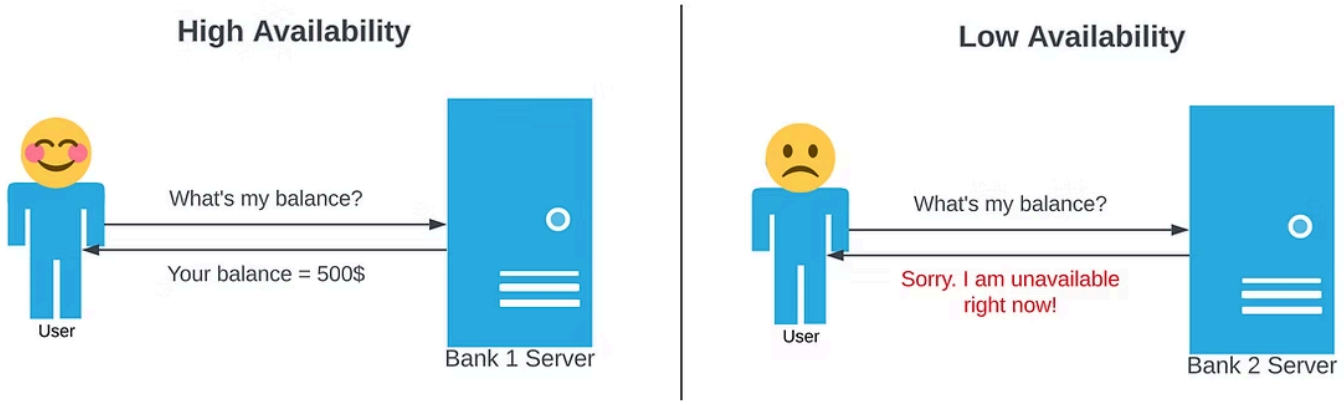
Availability means how much time a system is up or operational.

Example

For example, an online banking website that is available 24/7 ensures that users can access their accounts and perform transactions at any time.

High Availability

A system that is available **99.999%** of the time, also known as "*five nines,*" means it is only allowed **5 minutes (0.001%)** of downtime per year.



DESIGN GOALS :Consistency -Strong or Eventual

Consistency means consistent/same data visible to everyone.

For example, if you update your profile picture, every user sees the updated picture, and nobody sees the old one.

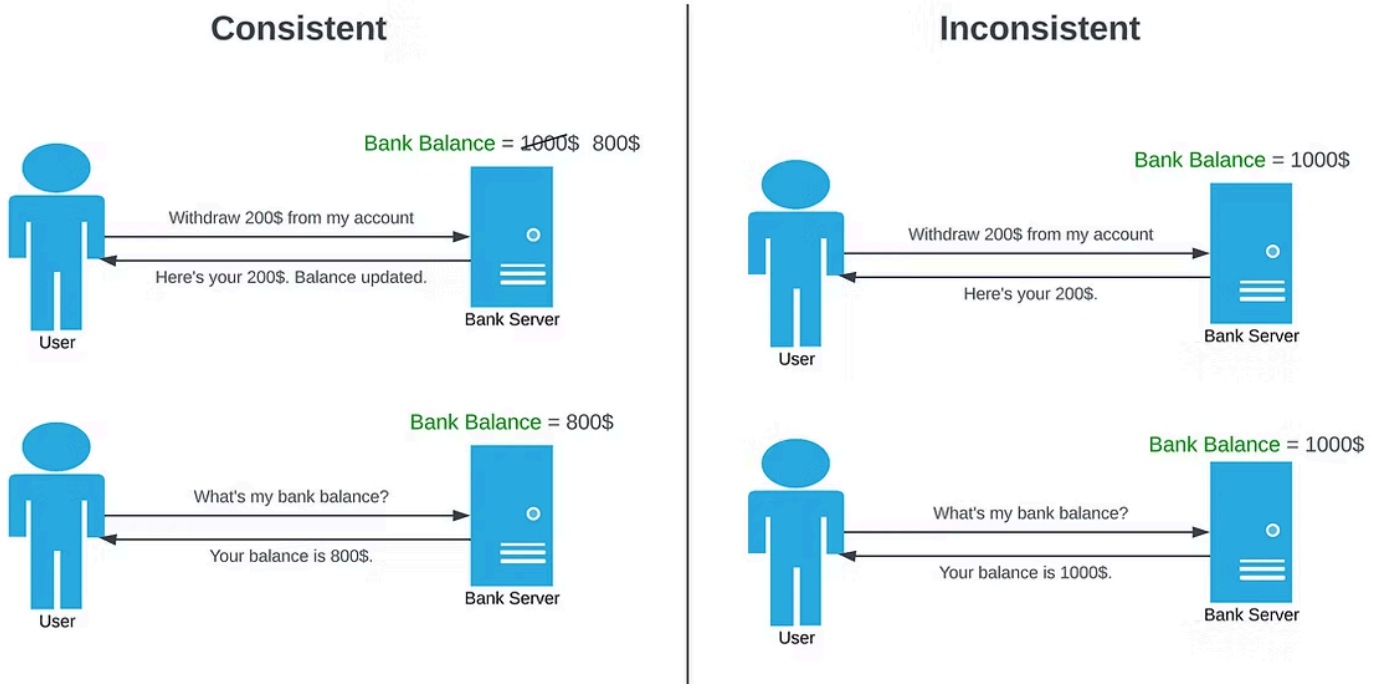
Ensuring consistency is important for applications where we need up-to-date information. One good example is financial transactions.

When you withdraw money from the bank, it’s essential that the updated balance is immediately reflected so the same money isn't withdrawn multiple times.

Strong Consistency

Strong Consistency means consistent/same data visible to everyone instantly, without any delay.

For example, when withdrawing money from the bank, it's essential that the updated balance is immediately reflected so the same money isn't withdrawn multiple times.

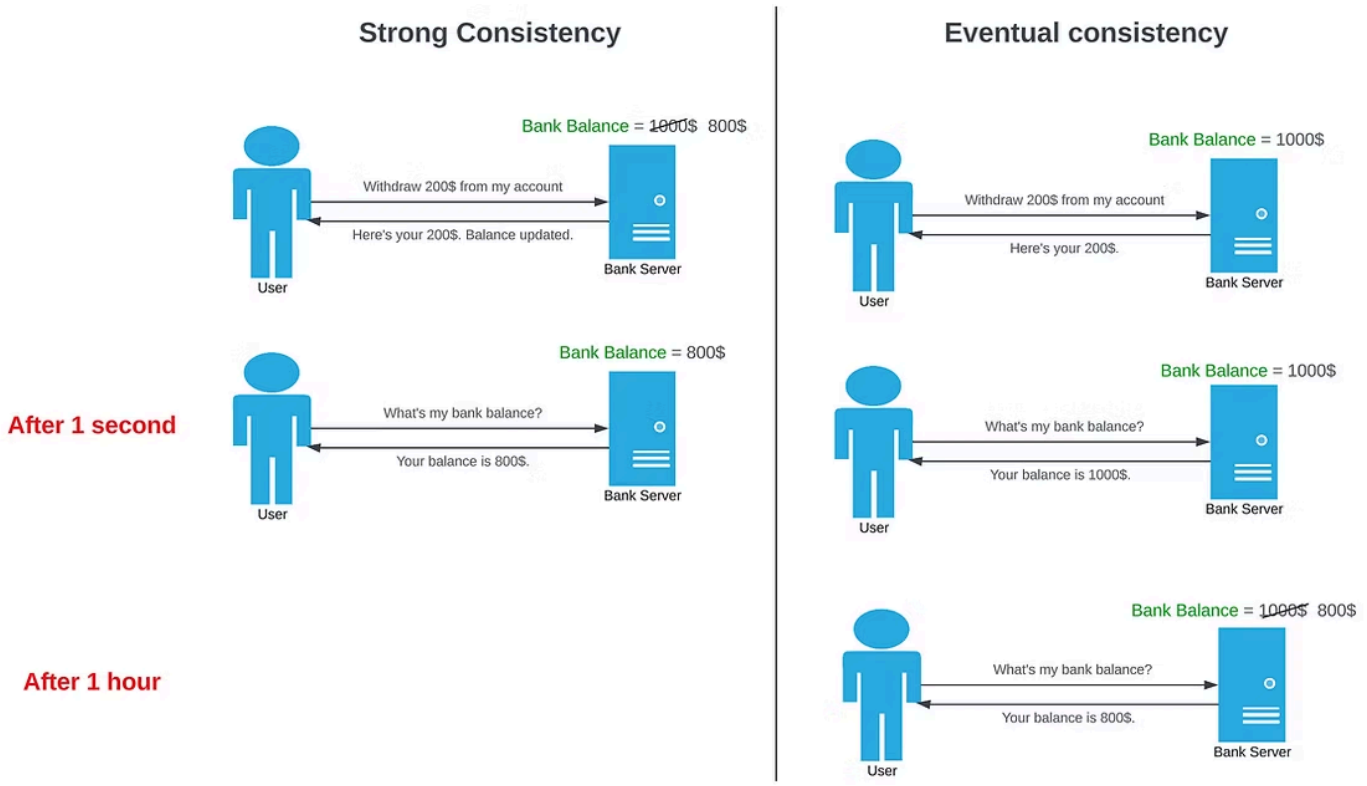


Eventual Consistency

Eventual Consistency means data will become consistent over time, but not necessarily instantly.

For example, if you update your profile picture, some users might still see the old picture for a short while, but eventually, everyone will see the updated one.

Eventual consistency can be used on social media or in applications where there is no strong need for instant updates.



DESIGN GOALS :Fault Tolerance and Single Point of Failure

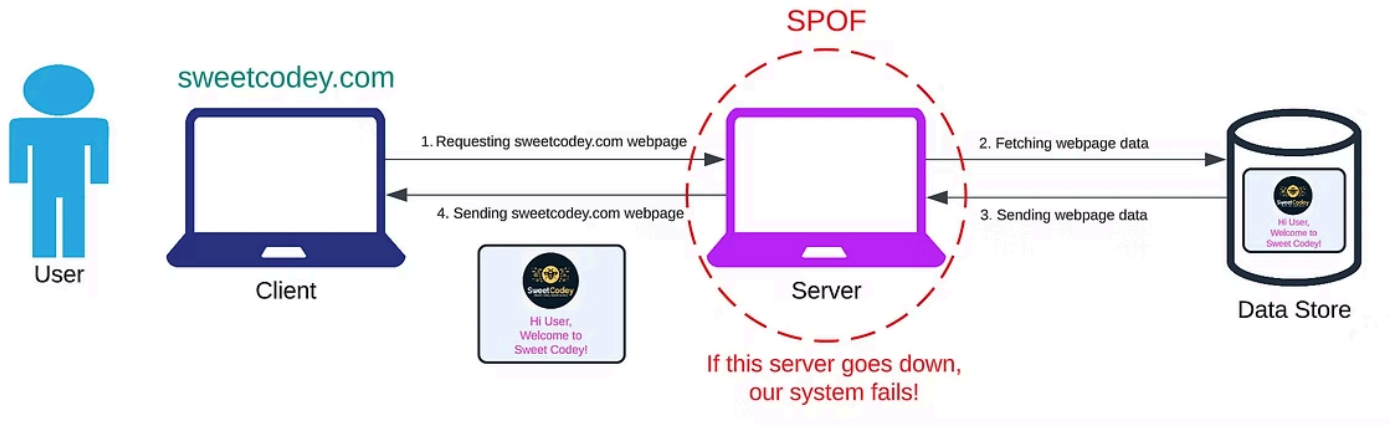
Let's assume a very simple system where a user is trying to open **sweetcodey.com**.

There is a **server** that handles the client's request and a **data store** that keeps the site data.

We can clearly see that if this server goes down, the website will be inaccessible. Here, this server is a **Single Point of Failure (SPOF).**

What is SPOF?

SPOF (Single Point of Failure) is a component in a system that, if it fails, will stop the entire system from working.



Avoiding SPOF

We can avoid **SPOF** in our system by adding more servers (aka adding **redundancy**).

This would make our system more **tolerant to failures**, as it will continue to operate even if some parts or components fail.

Technically, we say that we have made our system **Fault Tolerant**.

Example of a Fault Tolerant System:

- If **one server** in the data center fails, **another server** takes over.
- If **one data center** fails, **another data center** takes over.